Boundary Detection

edge detection → thresholding → shrink & expand → thinning → boundary detection

Fit line to edges

Given points (x_i, y_i). Find m,c for $y = mx + c$

- minimize avg. squared vert. distance

$$E = \frac{1}{N}\sum (y_i - mx_i - c)^2$$

$$\frac{\partial E}{\partial m} = \frac{-2}{N}\sum x_i(y_i - mx_i - c) = 0 \qquad \frac{\partial E}{\partial c} = -\frac{2}{N}\sum (y_i - mx_i - c) = 0$$

$$m = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sum (x_i - \bar{x})^2} \qquad c = \bar{y} - m\bar{x}$$

where $\bar{x} = \frac{1}{N}\sum x_i$, $\bar{y} = \frac{1}{N}\sum y_i$

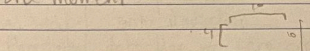
problem: bad for vertical line — minimizes line that E is perpendicular



Use $x\sin\theta - y\cos\theta + \rho = 0$

$$E = \frac{1}{N}\sum (x_i \sin\theta - y_i \cos\theta + \rho)^2 \rightarrow \text{perpendicular distance}$$

Treat like binary img and find axis of min. 2nd moment



$y = (x - x_0)\tan\theta + y_0$

$= x\tan\theta + (y_0 - x_0\tan\theta)$

Fitting Curves to Edges

easier: find polynomial $f(x) = ax^3 + bx^2 + cx + d$ w/ (x_i, y_i)

vertical dist: $E = \frac{1}{N}\sum (y_i - ax_i^3 - bx_i^2 - cx_i - d)^2$

solve lin sys:

$\frac{\partial E}{\partial a} = 0 \quad \frac{\partial E}{\partial b} = 0 \quad \frac{\partial E}{\partial c} = 0 \quad \frac{\partial E}{\partial d} = 0$

but as polynomial order ↑, this gets complex

$y_0 = ax_0^3 + bx_0^2 + cx_0 + d$

$y_1 = ax_1^3 + bx_1^2 + cx_1 + d$

$\vdots$

$y_i = ax_i^3 + bx_i^2 + cx_i + d$

$\vdots$

$y_n = ax_n^3 + bx_n^2 + cx_n + d$

w/ m unknowns, n observations ($n \geq m$), we do:

$$\begin{bmatrix} x_{00} & x_{01} & \cdots & x_{0m} \\ x_{10} & x_{11} & \cdots & x_{1m} \\ \vdots & \vdots & & \vdots \\ x_{n0} & x_{n0} & & x_{nm} \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ \vdots \\ a_m \end{bmatrix} = \begin{bmatrix} y_0 \\ y_1 \\ \vdots \\ y_n \end{bmatrix}$$

$X_{n\times m}$ (known) $\quad a_{m\times 1}$ (unknown) $\quad y_{n\times 1}$ (known)

but X isn't square → not invertible → multiply by X^T

X^TX is square & invertible

$$X^TXa = X^Ty \rightarrow a = (X^TX)^{-1}X^Ty \qquad X^+ = (X^TX)^{-1}X^T \text{ (pseudo inverse)}$$

Active Contours/Snakes

- powerful bc they can deform over time or with viewpoint

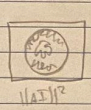
Representing contour

- contour v: ordered list of 2D vertices (control points) connected by straight lines of fixed length

$v = \{v_i = (x_i, y_i) \mid i = 0, 1, 2, \ldots, n-1\}$

Attracting to edges

- "trigger" w/ grad. mag. squared
- but first blur gradient so that points far away are drawn towards boundary



$\|\nabla I\|^2$



$\|\nabla_{\sigma} * I\|^2$

- Hum maximize sum of grad. magnitude squared
- = minimize -ve (sum of grad. mag. squared)
- = minimize $E_{image} = -\sum_{i=0}^{n} \|\nabla_{\sigma} * I(v_i)\|^2$

Greedy Algorithm

1. for each contour point v_i, move v_i to pos in window W where E_{image} is at min
2. if sum of motions of all contour points is less than threshold, stop. Else repeat

- sensitive to noise and the way the contor is initialized
- we want contours to be elastic and smooth

11:15

Minimize internal bending energy of contour

$E_{contour} = \alpha E_{elastic} + \beta E_{smooth}$

α and β control elasticity and smoothness

For point $0 \leq s \leq 1$ on cont. contour $v(s) = (x(s), y(s))$

$E_{elastic}: \left\|\frac{\partial v}{\partial s}\right\|^2 \qquad E_{smooth}: \left\|\frac{\partial^2 v}{\partial s^2}\right\|^2$

↓ we don't contour to stretch too fast — it's control points should be close together

↓ we don't want the dir of v too quickly

$$E_{elastic}(v_i) = \left\|\frac{\partial v}{\partial s}\right\|^2 \approx \|v_{i+1} - v_i\|^2 = (x_{i+1} - x_i)^2 + (y_{i+1} - y_i)^2$$

$$E_{smooth}(v_i) = \left\|\frac{\partial^2 v}{\partial s^2}\right\|^2 \approx \|(v_{i+1} - v_i) - (v_i - v_{i-1})\|^2$$

$$= (x_{i+1} - 2x_i + x_{i-1})^2 + (y_{i+1} - 2y_i + y_{i-1})^2$$

Internal bending energy: $E_{contour} = \alpha E_{elastic} + \beta E_{smooth}$

$= \alpha \sum E_{elastic}(v_i) + \beta \sum E_{smooth}(v_i)$

Total Energy: $E_{total} = E_{image} + E_{contour}$

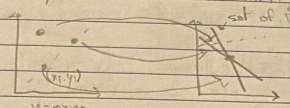
New Greedy Algorithm:

1. uniformly sample contour to get n contour points
2. for each point v_i, move in window where E_{total} is min
3. if sum of motions is over threshold, go back to step 1

Active Contours comments
- requires good initialization

Hough Transform
Difficulties for Fitting Approach
- Extraneous data
- Incomplete data
- Noise

Line Detection
- Given points (x_i, y_i), detect line $y = mx + c$
  $y_i = mx_i + c \leftrightarrow c = -mx_i + y_i$



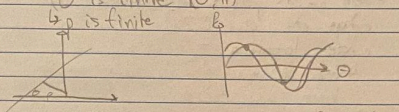
- extraneous points don't go through the common intercept

Algorithm
1. quantize parameter space (m, c)
2. create accumulator array $A(m, c)$
3. Set $A(m, c)$ to 0 for all (m, c)
4. For each edge point (x_i, y_i):
   $A(m, c) = A(m, c) + 1$
   if (m, c) lies on line $c = -mx_i + y_i$
5. Find local maxima in $A(m, c)$

"voting scheme"

$y = \frac{\rho}{\cos\theta}$

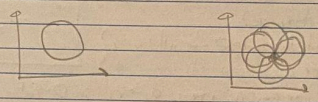
Better parameterization
- w/ m and c, $m \in (-\infty, \infty)$ but we can't have infinite accumulator
- better: $x\sin\theta - y\cos\theta + \rho = 0$
  - θ is finite $[0, \pi)$
  - ρ is finite



$x\sin\theta - y\cos\theta + \rho = 0$ $\quad$ $x\sin\theta - y\cos\theta + \rho = 0$
ing space

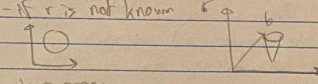
How big should accumulator?
- too big → lines could be merged
- too small → noise is issue

Circle Detection $(x_i - a)^2 + (y_i - b)^2 = r^2$
- if r is known



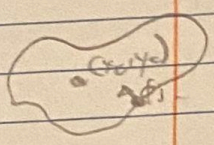
img space $\quad$ param. space
$(x_i - a)^2 + (y_i - b)^2 = r^2$ $\quad$ $(a - x_i)^2 + (b - y_i)^2 = r^2$

- if r is not known

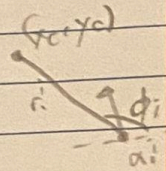


img space
$(x_i - a)^2 + (y_i - b)^2$ $\quad$ $(a - x_i)^2 + (b - y_i)^2 = r^2$

Generalized Hough Transform
for shapes that can't be described by equations

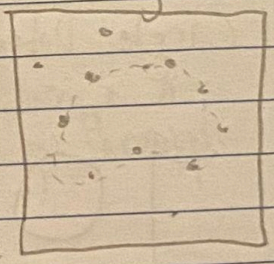


- reference point (x_c, y_c)
- for each point of boundary w/ edge dir $\phi_i \in [0, 2\pi]$
- edge location $\vec{r}_k^{\,i} = (r_k^i, \alpha_k^i)$
  ↳ may be multiple if many points have same dir ϕ_i



- R table

edge direction	$\vec{r} = (r, \alpha)$
ϕ_1	$\vec{r}_1^{\,1}, \vec{r}_2^{\,1}, \vec{r}_3^{\,1}$
ϕ_2	$\vec{r}_1^{\,2}, \vec{r}_2^{\,2}$
⋮	⋮
ϕ_n	$\vec{r}_1^{\,n}, \vec{r}_2^{\,n}, \vec{r}_3^{\,n}, \vec{r}_4^{\,n}$



Generalized Algorithm
1. Creat accumulator array $A(x_c, y_c)$
2. Set $A(x_c, y_c) = 0$ for all (x_c, y_c)
3. For each edge pt. (x_i, y_i, ϕ_i):
   - For each entry $\phi_i \to \vec{r}_k^{\,i}$ in ϕ-table
     $x_c = x_i \pm r_k^i \cos(\alpha_k^i)$
     $y_c = y_i \pm r_k^i \sin(\alpha_k^i)$
     $A(x_c, y_c) = A(x_c, y_c) + 1$

Handling Scale and Rotation
- use Acc. Array $A(x_c, y_c, s, \theta)$